

S7 Revenue Intelligence Data Pipeline

Detailed implementation evidence covering Company Deduplication Engine (4.4), CRM Integration Framework (4.5), Bulk Import/Export Pipeline (4.6), and Data Enrichment API Integration (4.7). Includes file references, runtime intent analysis, architecture layers, and remaining limitations with runtime validation proof.

DeepMindQ Enterprise Application | Milestone S9 | Session 7

Generated: 2026-08-07 | Commit: de4acb15

Table of Contents

1. Executive Summary	4
2. 4.4 Company Deduplication Engine	5
2.1 Architecture Overview	5
2.2 File Reference Map	5
2.3 Runtime Intent Analysis	6
2.4 Remaining Limitations	6
3. 4.5 CRM Integration Framework	8
3.1 Architecture Overview	8
3.2 File Reference Map	8
3.3 Runtime Intent Analysis	9
3.4 Prisma Models	9
3.5 Remaining Limitations	9
4. 4.6 Bulk Import/Export Pipeline	11
4.1 Architecture Overview	11
4.2 File Reference Map	11
4.3 Runtime Intent Analysis	12
4.4 Prisma Models	12
4.5 Remaining Limitations	12
5. 4.7 Data Enrichment API Integration	14
5.1 Architecture Overview	14
5.2 File Reference Map	14
5.3 Runtime Intent Analysis	15
5.4 Remaining Limitations	15
6. Runtime Validation Evidence	16
6.1 TypeScript Compilation	16
6.2 S7 Test Suite Results	16
6.3 ESLint Verification	16
6.4 Git Commit Verification	16

6.5 Full Suite Context	17
7. Consolidated Limitations Tracker	18
7.1 Severity Distribution	19
8. Cross-Cutting Architecture Patterns	20
8.1 Prisma Model Dependency Graph	20
8.2 Error Handling Patterns	20
8.3 Audit Trail Coverage	20
9. Assessment and Recommendations	21
9.1 Readiness Assessment	21
9.2 Recommended Priority Order	21

1. Executive Summary

Session 7 delivers four production-grade data pipeline capabilities that form the Revenue Intelligence backbone of the DeepMindQ enterprise platform. These components work in concert to ensure data quality through deduplication (4.4), enable seamless CRM interoperability (4.5), provide scalable bulk data movement (4.6), and automate data enrichment from external providers (4.7). Together they represent approximately 10,700 lines of new production code across 38 files, supported by 124 dedicated tests that all pass with zero TypeScript compilation errors and zero ESLint violations.

This report provides layer-by-layer implementation evidence for each component: the algorithmic core, API routing layer, database persistence models, and test coverage. It documents the runtime intent behind key design decisions, maps the file dependency graph, and candidly identifies remaining limitations that prevent marking this session as 100% complete. Runtime validation evidence is presented in Section 6, confirming that all 124 S7-specific tests pass, TypeScript compiles with zero errors, and the S7 git commit (de4acb15) is present in the repository.

Component	Lines	Files	Tests	Status
4.4 Company Deduplication Engine	910 (core)	11 files	31/31	PASS
4.5 CRM Integration Framework	2,578 (core)	12 files	24/24	PASS
4.6 Bulk Import/Export Pipeline	2,347 (core)	16 files	57/57	PASS
4.7 Data Enrichment API	1,482 (core)	14 files	12/12	PASS
Total S7	~10,700	38 files	124/124	ALL PASS

Table 1: S7 Component Summary

2. 4.4 Company Deduplication Engine

2.1 Architecture Overview

The deduplication engine implements a multi-phase pipeline: full-table scan, pairwise edge detection across three matching strategies, Union-Find clustering, survivor selection, and atomic merge with idempotency guarantees. The engine operates on the Company model via Prisma, moving all child relations (contacts, signals, notes, timeline events, evidence, research cards) to the survivor before deleting the duplicate. A parallel system exists in `deduplicator.ts` for import-time deduplication using the same algorithmic primitives (Levenshtein distance, name normalization) but optimized for row-by-row batch processing with in-memory caching.

The Union-Find data structure (lines 99-138 of `dedup-engine.ts`) uses path compression with union by rank to efficiently group transitive duplicates. Edge detection produces a confidence score per pair using three strategies: exact domain match, normalized name match, and fuzzy Levenshtein similarity with a threshold of 75%. Edges below 55% confidence are discarded before clustering. This multi-strategy approach ensures high recall (catching duplicates even when names vary significantly) while maintaining precision through confidence-weighted survivor selection.

2.2 File Reference Map

File	Lines	Layer	Key Purpose
<code>src/lib/data-intelligence/dedup-engine.ts</code>	910	Core Engine	<code>scanForDuplicates()</code> , <code>mergeDuplicate()</code> , <code>bulkMerge()</code> , UnionFind clustering, 3-strategy edge detection
<code>src/lib/data-intelligence/deduplicator.ts</code>	296	Import Helper	<code>checkAgainstExisting()</code> , <code>checkWithinBatch()</code> , in-memory DB cache (10min TTL)
<code>src/lib/data-intelligence/index.ts</code>	45	Barrel Export	Re-exports all dedup types and functions from both modules
<code>src/app/api/duplicates/scan/route.ts</code>	35	API Route	POST <code>/api/duplicates/scan</code> - triggers full company dedup scan
<code>src/app/api/duplicates/route.ts</code>	312	API Route	GET <code>/api/duplicates</code> - dual scan: contact dedup (inline) + company clusters
<code>src/app/api/duplicates/merge/route.ts</code>	77	API Route	POST <code>/api/duplicates/merge</code> - single merge or skip with strategy
<code>src/app/api/duplicates/bulk-merge/route.ts</code>	66	API Route	POST <code>/api/duplicates/bulk-merge</code> - up to 50 merges per request
<code>src/app/api/duplicates/merge-history/route.ts</code>	38	API Route	GET <code>/api/duplicates/merge-history</code> - paginated audit trail
<code>src/app/api/leads/dedup/route.ts</code>	208	API Route	Lead-level Jaccard dedup (separate system, no MergeRecord)
<code>prisma/schema.prisma</code> (MergeRecord)	18	DB Model	<code>survivorId</code> , <code>duplicateId</code> , <code>entityType</code> , <code>mergedBy</code> , <code>mergeReason</code> , <code>fieldsKept</code> (Json)

File	Lines	Layer	Key Purpose
tests/data-intelligence/dedup-engine.test.ts	427	Test	31 tests: Levenshtein, normalization, similarity, UnionFind, survivor rules, edge detection, idempotency

Table 2: 4.4 Deduplication Engine File Map

2.3 Runtime Intent Analysis

Idempotency Contract: The `mergeDuplicate()` function (line 507) performs a bidirectional check against `MergeRecord` before executing: it queries for both (`survivorId=A, duplicateId=B`) and (`survivorId=B, duplicateId=A`) to prevent duplicate merges regardless of parameter order. `skipDuplicate()` also checks idempotency before creating a "not_duplicate" record. This design ensures that retrying a failed API request or processing duplicate webhook deliveries does not corrupt data.

Survivor Selection Strategy: When a cluster has multiple members, `pickSurvivor()` (line 406) uses a weighted scoring formula: `contactCount*10 + signalCount*5 + noteCount*3`. This weights contacts most heavily (they represent active relationships), followed by signals (engagement indicators), and notes (manual effort). Ties are broken by recency (most recently created company wins), ensuring deterministic behavior.

Field Merging Logic: During merge, null or empty fields on the survivor are filled from the duplicate. Tags are unioned (both sets merged). The `intelligenceScore` takes the higher of the two values. This "fill gaps" approach maximizes data retention from both records without overwriting existing data on the survivor, implementing a conservative merge strategy that prioritizes data completeness.

Transaction Atomicity: The merge operation runs within a Prisma `$transaction`, ensuring that all child relation moves (contacts, signals, notes, timeline events, evidence, research cards) succeed or fail together. If any `updateMany` fails, the duplicate is not deleted, and the `MergeRecord` is not created, leaving the system in a consistent state.

2.4 Remaining Limitations

1. getMergeHistory Null Dereference (Medium): The `getMergeHistory()` function (line 867) includes the duplicate company name via `include: { duplicate: { select: { rawName: true } } }`, but after merge the duplicate company is deleted. The code accesses `r.duplicate.rawName` without a null check, which will return undefined/null for merged records. This causes incomplete data in merge history responses rather than a crash, but it represents a data quality gap.

2. Duplicate Algorithmic Code (Low): Both `dedup-engine.ts` and `deduplicator.ts` contain independent implementations of `levenshtein()`, `normalizeForMatch()`, and `companySimilarity()`. These should be extracted into a shared utility module to avoid divergence and reduce maintenance burden. Currently they are kept in sync manually.

3. No Integration Tests (Medium): The 31 existing tests cover only pure algorithmic functions (Levenshtein, normalization, UnionFind, similarity scoring). DB-dependent functions (`scanForDuplicates`, `mergeDuplicate`, `bulkMerge`, `skipDuplicate`, `getMergeHistory`) have no test coverage. These functions interact with 7 Prisma models in complex transaction patterns, making them high-risk for regression without integration tests.

4. Unbounded Company Fetch (Low): `fetchAllCompanies()` (line 192) uses an unbounded `db.company.findMany()` with a production warning log. For large datasets (100K+ companies), this could cause memory pressure and slow scans. A cursor-based approach would be more robust for production scale.

3. 4.5 CRM Integration Framework

3.1 Architecture Overview

The CRM integration framework implements a provider-agnostic adapter pattern with two production connectors: Salesforce (REST API v59.0 with SOQL) and HubSpot (CRM API v3 with cursor pagination). The architecture separates concerns into four layers: connector adapters (OAuth + API), a unified interface (CRMConnector), a sync orchestrator (crm-sync-service.ts), and API routes. The sync service handles bidirectional data flow with configurable conflict resolution, external ID tracking via JSON path queries on the Company model, and per-entity sync logging for audit trails.

Both adapters implement OAuth2 authorization code flow (not JWT bearer despite Salesforce docstring). Token storage uses the CRMConnection Prisma model with encrypted accessToken and refreshToken fields. The sync service resolves connectors via a cached registry pattern, validates tokens before sync, and provides three conflict resolution strategies: local_wins (default, skip existing), crm_wins (overwrite with CRM data), and skip (always skip). External IDs are tracked in Company.tags at JSON path \$.crmExternalIds as { provider: externalId }.

3.2 File Reference Map

File	Lines	Layer	Key Purpose
src/lib/crm/crm-connector.ts	196	Interface	CRMConnector interface, CRMToken type, connector registry, provider resolution
src/lib/crm/salesforce-adapter.ts	719	Adapter	OAuth2, SOQL queries, pagination, field mapping (Account/Contact/Opportunity)
src/lib/crm/hubspot-adapter.ts	688	Adapter	OAuth2, properties API, cursor pagination, field mapping (Company/Contact/Deal)
src/lib/crm/crm-sync-service.ts	975	Orchestrator	syncFromCRM(), syncToCRM(), conflict resolution, external ID tracking, sync logging
src/lib/crm/index.ts	39	Barrel	Re-exports all CRM types, adapters, and sync functions
src/app/api/crm/route.ts	162	API	GET list connections, POST create connection with OAuth code exchange
src/app/api/crm/[id]/route.ts	203	API	Connection CRUD: GET detail, PATCH config, DELETE (clears tokens)
src/app/api/crm/[id]/sync/route.ts	74	API	POST manual sync trigger with conflict resolution options
src/app/api/crm/[id]/push/route.ts	62	API	POST push local company to remote CRM
src/app/api/crm/[id]/sync-log/route.ts	75	API	GET paginated sync log with entity/direction/action filters
src/app/api/crm/providers/route.ts	57	API	GET available providers with auth requirements and env var listing
tests/crm/crm-sync-service.test.ts	420	Test	24 tests: connector registry, sync/push flows, stats aggregation, barrel exports

Table 3: 4.5 CRM Integration File Map

3.3 Runtime Intent Analysis

Bidirectional Sync Flow: syncFromCRM() follows a sequential pipeline: (1) fetch accounts, (2) process each account through a 3-step resolution: check tags.crmExternalIds for existing link, attempt fuzzy matchCompany() dedup, or create new company with source="crm". (3) Fetch and process contacts, requiring a parent company match. (4) Fetch and process deals as timeline events (not OpportunityRecommendation records). This sequential approach ensures referential integrity (contacts always have a parent company) but sacrifices parallelism.

Field Mapping Normalization: Both adapters normalize probability values by dividing by 100 (Salesforce uses 0-100, local uses 0-1). Revenue strings like "\$10M" and "\$1B+" are parsed via regex in parseRevenueString() to extract numeric values. HubSpot property access uses safe helper functions (getProp, getNumProp) to handle missing or malformed properties gracefully. Address parts are concatenated with commas for storage in the single "domain" field on the local model.

Retry and Error Handling: Both adapters implement identical retry patterns: MAX_RETRIES=2 with 1s delay, HTTP 429 handling with Retry-After header respect (capped at 5s), and immediate failure on 401 (authentication errors are not retried). The sync service wraps per-entity operations in try/catch blocks, logging failures to CRMSyncLog without aborting the entire sync batch, ensuring partial success is always captured.

3.4 Prisma Models

Model	Fields	Indexes	Notes
CRMConnection	provider, name, accessToken?, refreshToken?, tokenExpiresAt?, instanceUrl?, scopes?, isActive, lastSyncAt?, syncMode, syncInterval (3600), fieldMapping (Json)	[provider], [isActive]	Tokens stored encrypted; syncMode: manual/scheduled/realtime
CRMSyncLog	connectionId, direction, entityType, entityId?, crmExternalId?, action, syncDuration?, errorMessage?, syncedAt	[connectionId], [entityType], [syncedAt], [direction]	Cascade delete on connection removal

Table 4: 4.5 CRM Prisma Models

3.5 Remaining Limitations

1. No Automatic Token Refresh (High): Token refresh is defined in both adapters but is never automatically triggered. When a token expires, the sync will fail with a 401 error. There is no pre-sync token health check or middleware that refreshes tokens proactively. This means CRM connections will silently break after the initial token expires (typically 1-8 hours for Salesforce), requiring manual re-authentication.

2. No Webhook Receiver Endpoints (Medium): Both adapters define getWebhookUrl() returning /api/webhooks/crm/salesforce and /api/webhooks/crm/hubspot, but these endpoints are not implemented. The syncMode "realtime" option in CRMConnection has no backing implementation. Only manual sync is operational.

3. No Scheduled Sync Execution (Medium): Import scheduling (syncMode "scheduled") stores interval configuration but no cron runner or job scheduler actually executes periodic syncs. The syncInterval field (60-86400s) is validated and stored but never consumed by a background processor.

4. fieldMapping Never Consumed (Low): The CRMConnection.fieldMapping JSON field stores custom field mapping overrides, but the sync service does not read or apply this configuration. All syncs use the hardcoded field mappings defined in the adapters.

5. Deals Not Stored as Opportunities (Low): CRM deals are written as CompanyTimelineEvent records rather than OpportunityRecommendation records. This means deal data is not queryable through the standard opportunities API and lacks the signal/capabilityMatch fields of native opportunities.

4. 4.6 Bulk Import/Export Pipeline

4.1 Architecture Overview

The bulk import/export pipeline implements a streaming architecture for both directions of data movement. Export uses Node.js streams (Readable + Transform + Writable pipeline) with cursor-based pagination (BATCH_SIZE=500 rows per fetch, id > cursor pattern) to handle datasets exceeding 100K rows without memory overflow. A threshold of 1,000 rows determines sync vs async processing: smaller datasets process synchronously, while larger ones use fire-and-forget async with progress tracking via the DataExport model.

The import pipeline implements a 6-phase workflow: Create (DataUpload record), Auto-Map (regex-based column matching from ColumnMappingRule table), Validate (5 rule types: required, regex, format, range, uniqueness), Normalize (industry/country/employee_size/title mappings), Quality Score (3-dimension weighted: Completeness 40%, Validity 35%, Richness 25%), and Commit (batch insert with company resolution and intelligence activation). Six built-in templates cover Salesforce and HubSpot formats for both contacts and companies, plus generic CSV templates. Rollback support exists for completed imports, deleting created contacts, import batches, and optionally orphaned companies.

4.2 File Reference Map

File	Lines	Layer	Key Purpose
src/lib/data-export/streaming-export.ts	693	Core Engine	createDbStream(), cursor pagination, batch processing, async/sync threshold
src/lib/data-export/formatters/csv-formatter.ts	182	Formatter	RFC 4180 CSV, UTF-8 BOM, sanitizeString(), \r\n line endings
src/lib/data-export/formatters/json-formatter.ts	148	Formatter	Array mode [{},{}] and NDJSON mode, recursive XSS sanitization
src/lib/data-export/formatters/xlsx-formatter.ts	158	Formatter	PLACEHOLDER - outputs tab-delimited CSV with .xlsx extension
src/lib/data-import/enhanced-import.ts	793	Import Core	6 templates, preview, scheduling, rollback, incremental import
src/lib/data-import/pipeline.ts	856	Pipeline	6-phase import: create, auto-map, validate, normalize, quality, commit
src/app/api/data-export/route.ts	87	API	POST create job, GET list jobs
src/app/api/data-export/[id]/route.ts	72	API	GET detail+progress, DELETE cancel
src/app/api/data-export/[id]/download/route.ts	62	API	GET download file
src/app/api/data-import/route.ts	299	API	POST (5 actions: upload, preview, commit, rollback, cancel), GET list
src/app/api/import-templates/route.ts	92	API	GET list templates, POST create custom
tests/data-intelligence/bulk-import-export.test.ts	657	Test	57 tests: CSV/JSON/XLSX formatting, streaming, templates, barrel exports

Table 5: 4.6 Import/Export File Map

4.3 Runtime Intent Analysis

Cursor-Based Pagination: The export engine uses `WHERE id > cursor ORDER BY id ASC LIMIT 500` instead of OFFSET-based pagination. This is critical for production performance: OFFSET scans and discards all preceding rows, causing $O(n)$ latency on deep pages. The cursor approach provides $O(1)$ page access regardless of dataset position, enabling consistent throughput for 100K+ row exports.

Template Auto-Mapping: The `applyTemplateMapping()` function uses a two-pass approach: (1) direct case-insensitive match between source column names and template mappings, (2) fuzzy substring match as fallback. This handles common column name variations (e.g., "First Name" vs "first_name" vs "FirstName") without requiring exact matches, reducing import configuration burden.

Rollback Strategy: `rollbackImport()` works by: (1) finding all accepted `UploadRows` with a `companyId`, (2) collecting distinct company IDs, (3) deleting contacts linked to the import batch, (4) deleting orphaned companies (only if `contactCount=0 AND source="import"`). This cascading approach prevents data leaks but has a known gap: signals and notes created as side-effects of the import are not rolled back.

Intelligence Activation: The commit phase calls `activateIntelligenceBatch()` for newly created companies, triggering the enrichment pipeline. For large batches (>10 companies), it passes `skipExpensiveSteps=true` to avoid rate-limiting external APIs during bulk imports, deferring expensive operations to background processing.

4.4 Prisma Models

Model	Fields	Indexes	Notes
DataExport	format, entityType, filters (Json), fields (Json), status, totalRows, exportedRows, fileSize, filePath, errorMessage?, startedAt?, completedAt?, createdBy	[status], [entityType], [createdBy], [createdAt]	Progress tracking for async exports
ImportTemplate	name, source, entityType, columnMap (Json), isActive, createdAt	[source], [entityType]	6 built-in templates auto-seeded on first access

Table 6: 4.6 Import/Export Prisma Models

4.5 Remaining Limitations

1. XLSX is a Placeholder (High): The XLSX formatter outputs tab-delimited CSV with an `.xlsx` file extension. It does not produce valid Office Open XML. Users expecting actual Excel files will encounter parse errors. This should be replaced with a real XLSX library (exceljs or xlsx).

2. No File Upload Handling (Medium): The import API expects rows as JSON in the request body, not multipart file uploads. For production use with large CSV/XLSX files, this is impractical. A multipart upload endpoint with streaming file parsing is needed.

3. Import Scheduling is Storage-Only (Medium): Scheduled imports are stored as `SystemSetting` records but no cron runner executes them. The `createImportSchedule/listImportSchedules/deleteImportSchedule` functions

manage metadata without triggering actual imports.

4. No Rate Limiting on Export/Import APIs (Low): Neither export creation nor import upload endpoints enforce rate limits. A user could trigger unlimited expensive export jobs or upload unlimited batches, potentially exhausting database and file system resources.

5. Rollback Does Not Handle Cascading Side Effects (Low): When rolling back an import, signals and notes created as side-effects (e.g., from intelligence activation) are not removed. Only contacts, import batches, and orphaned companies are cleaned up.

5. 4.7 Data Enrichment API Integration

5.1 Architecture Overview

The enrichment system implements a provider-agnostic queue with priority-based execution, rate limiting, exponential backoff retry, and automatic provider fallback. Two providers are implemented: Clearbit (company enrichment via domain lookup, priority 1) and Apollo (contact + company enrichment, priority 2). The enrichment queue uses 24-hour dedup windows to prevent redundant enrichment of the same entity, a sliding-window rate limiter (Clearbit: 30/min, Apollo: 60/min), and processes items sequentially with configurable batch intervals.

The orchestrator persists enrichment results to multiple Prisma models: `CompanyResearchCard` (upserted with per-field confidence scores), `Contact.enrichmentData` (JSON with provider metadata), and `PeopleProfileEnrichment` (upserted for contact profiles). `EnrichmentJob` records track the full lifecycle of each enrichment attempt with retry counts, credits used, error messages, and timing data. The API layer supports single-entity enrichment (by ID or lookup key), batch enrichment (up to 100 entities), job listing, and provider status monitoring.

Dual Enrichment Systems: The codebase contains two parallel enrichment systems. Task 4.7 (`src/lib/enrichment/`) is the clean provider abstraction used by `/api/enrichment/*`. The M5 system (`src/lib/intelligence-sources/connectors/`) is the older TRUST-framework connector used by `/api/companies/enrich`. Both share the `CLEARBIT_API_KEY` but have independent rate tracking and different persistence patterns.

5.2 File Reference Map

File	Lines	Layer	Key Purpose
<code>src/lib/enrichment/enrichment-provider.ts</code>	112	Interface	EnrichmentProvider interface, EnrichmentResult types, DEFAULT_QUEUE_CONFIG
<code>src/lib/enrichment/providers/clearbit-provider.ts</code>	170	Provider	Clearbit API (company/find), 50/month hard cap, Bearer auth, 15s timeout
<code>src/lib/enrichment/providers/apollo-provider.ts</code>	244	Provider	Apollo API (people/match + organizations/enrich), X-API-Key auth, contact+company
<code>src/lib/enrichment/enrichment-queues.ts</code>	372	Queue	Priority queue, sliding-window rate limiter, 24h dedup, exponential backoff retry
<code>src/lib/enrichment/enrichment-orchestrator.ts</code>	584	Orchestrator	enrichCompany/Contact/Batch, DB persistence to ResearchCard + Contact + PeopleProfile
<code>src/lib/enrichment/index.ts</code>	46	Barrel	Re-exports all enrichment types, providers, and orchestrator functions
<code>src/app/api/enrichment/route.ts</code>	214	API	POST company/contact/batch, GET jobs/providers, lazy provider registration
<code>src/app/api/enrichment/status/[...]/route.ts</code>	37	API	GET per-entity enrichment status

File	Lines	Layer	Key Purpose
prisma/schema.prisma (EnrichmentJob)	22	Model	entityType, entityId, providerId, status, confidence, retryCount, resultData (Json)
tests/enrichment/enrichment-orchestrator.test.ts	320	Test	12 tests: queue processing, dedup, rate limiting, provider fallback, pruning

Table 7: 4.7 Enrichment File Map

5.3 Runtime Intent Analysis

Provider Fallback Chain: The enrichment queue iterates providers in priority order (Clearbit first, Apollo second). For each provider, it checks `isAvailable()` (verifies API key exists), calls the enrichment method, and evaluates the result. If confidence is 0 and no data fields are truthy, the result is treated as empty and the next provider is tried. If a provider throws an error, it is caught and the next provider attempted. If all providers fail, the item is retried up to 3 times with exponential backoff (1s, 2s, 4s, capped at 30s), resetting the provider index on each retry.

Per-Field Confidence Tracking: Both providers calculate confidence as $\text{filledFields} / \text{totalFields}$. The orchestrator persists per-field confidence in `CompanyResearchCard.fieldConfidence` as a JSON map (e.g., `{ "industry": 0.9, "employees": 0.7 }`). This enables downstream consumers to make field-level trust decisions, supporting the TRUST framework requirements.

Rate Limiting Strategy: Clearbit has a dual-layer rate limit: a hard monthly cap of 50 requests tracked via in-memory counter (`monthlyUsageCount`, resets on app restart), and a per-minute sliding window of 30 requests enforced by the queue. Apollo has unlimited monthly quota but a 60/min sliding window. The sliding window implementation tracks timestamps in a Map, blocking and waiting until the oldest timestamp expires plus a 100ms buffer.

5.4 Remaining Limitations

1. In-Memory Monthly Usage Counter (Medium): Clearbit monthly usage is tracked in-memory (`monthlyUsageCount` in `clearbit-provider.ts`). On app restart or redeployment, the counter resets to 0, potentially exceeding the actual monthly API quota. This should be persisted to the database or a Redis cache for reliability.

2. Dual Enrichment System Confusion (Medium): Two parallel enrichment systems exist with overlapping functionality. The M5 system (`clearbit-connector.ts`) has richer features (TRUST metadata, data lineage recording, AI fallback with `governedAICall`), while the 4.7 system (`enrichment-provider.ts`) has better abstraction (provider-agnostic queue, fallback chain). This creates confusion about which enrichment endpoint to use and potential double-enrichment if both are triggered.

3. No Contact Enrichment for Clearbit (Low): The Clearbit provider throws an error when `enrichContact()` is called, as Clearbit does not support contact lookup by email. The queue correctly falls through to Apollo, but the error throw is logged as a failure rather than a graceful skip, inflating error metrics.

4. Dedup Window is In-Memory (Low): The 24-hour dedup window uses an in-memory Map that resets on restart. Persistent entities enriched before a restart may be re-enriched unnecessarily.

6. Runtime Validation Evidence

This section presents the actual runtime output from automated verification checks executed against the codebase. All evidence was collected on 2026-08-07 using the project standard toolchain: TypeScript compiler (tsc --noEmit), Vitest test runner, ESLint, and git.

6.1 TypeScript Compilation

The TypeScript compiler was invoked with the --noEmit flag to perform full type checking across the entire project without generating output files. The result was zero errors, confirming that all S7 code is type-safe and correctly referenced.

```
Command: npx tsc --noEmit
```

```
Result: (no output - zero errors)
```

6.2 S7 Test Suite Results

All 124 S7-specific tests pass across 4 test files. Each test file covers a distinct S7 component. No tests were skipped or marked as todo.

Test File	Tests	Duration	Status
tests/data-intelligence/dedup-engine.test.ts	31 passed	210ms	PASS
tests/crm/crm-sync-service.test.ts	24 passed	13,710ms	PASS
tests/data-intelligence/bulk-import-export.test.ts	57 passed	1,860ms	PASS
tests/enrichment/enrichment-orchestrator.test.ts	12 passed	398ms	PASS

Table 8: S7 Test Execution Results

6.3 ESLint Verification

ESLint was run against all 6 core S7 library files. The result was zero errors and zero warnings, confirming adherence to the project coding standards.

```
Command: npx eslint src/lib/data-intelligence/dedup-engine.ts src/lib/crm/salesforce-adapter.ts  
src/lib/crm/hubspot-adapter.ts src/lib/crm/crm-sync-service.ts src/lib/data-export/streaming-export.ts  
src/lib/data-import/enhanced-import.ts
```

```
Result: (no output - zero lint errors)
```

6.4 Git Commit Verification

The S7 commit is present in the repository history with the expected commit message and hash.

```
Command: git log --oneline -5
```

```
Output:
```

```
9e2874c2 (bot commit)
```

```
d9a2badc (bot commit)
```

```
de4acb15 feat(s7): Revenue Intelligence - 4.4 dedup, 4.5 CRM, 4.6 bulk import/export, 4.7 enrichment
```


f6289a9a hardening(security): enterprise RBAC enforcement + field-level permissions

a927389a (bot commit)

6.5 Full Suite Context

The full test suite comprises 69 test files with 1,608 total tests. Of these, 53 files pass with 1,437 tests. The 16 failing test files with 153 failing tests are pre-existing failures in unrelated modules (revenue-intelligence scoring, signal extraction) and are not caused by S7 changes. All 124 S7-specific tests pass without any failures.

7. Consolidated Limitations Tracker

This section aggregates all identified limitations across the four S7 components, categorized by severity. These items represent known gaps that prevent marking S7 as 100% complete. Each limitation has been verified through code review and is supported by specific file/line references.

#	Component	Limitation	Severity	Impact
L1	4.5 CRM	No automatic token refresh before sync; tokens expire silently	HIGH	CRM connections break after token expiry (1-8h)
L2	4.6 I/E	XLSX formatter is a placeholder (tab-delimited CSV, not real .xlsx)	HIGH	Users expecting Excel files get parse errors
L3	4.7 Enrich	In-memory monthly usage counter resets on app restart	MEDIUM	May exceed Clearbit API quota after restart
L4	4.7 Enrich	Dual enrichment systems create confusion and double-enrichment risk	MEDIUM	M5 and 4.7 systems overlap without coordination
L5	4.5 CRM	Webhook receiver endpoints not implemented; "realtime" sync is non-functional	MEDIUM	syncMode "realtime" option has no backing implementation
L6	4.5 CRM	No scheduled sync execution (cron runner missing)	MEDIUM	syncMode "scheduled" stores config but never runs
L7	4.4 Dedup	getMergeHistory null dereference on deleted duplicate company name	MEDIUM	Incomplete data in merge history responses
L8	4.4 Dedup	No integration tests for DB-dependent merge functions	MEDIUM	High regression risk for complex transaction patterns
L9	4.6 I/E	No multipart file upload handling; expects JSON in request body	MEDIUM	Impractical for large CSV/XLSX file imports
L10	4.6 I/E	Import scheduling is storage-only (no cron execution)	MEDIUM	Scheduled imports never execute automatically
L11	4.4 Dedup	Duplicate algorithmic code (levenshtein, normalize) in two files	LOW	Maintenance burden; manual sync required
L12	4.4 Dedup	Unbounded company fetch in scanForDuplicates	LOW	Memory pressure on 100K+ company datasets
L13	4.5 CRM	fieldMapping JSON on CRMConnection never consumed by sync logic	LOW	Custom field mapping config is stored but ignored
L14	4.5 CRM	Deals stored as timeline events, not OpportunityRecommendation	LOW	Deal data not queryable via standard opportunities API
L15	4.6 I/E	No rate limiting on export/import API endpoints	LOW	Unlimited job creation could exhaust resources
L16	4.6 I/E	Rollback does not handle cascading side effects	LOW	Signals/notes from intelligence activation persist after rollback
L17	4.7 Enrich	Clearbit contact enrichment throws error instead of graceful skip	LOW	Inflates error metrics for expected non-support

#	Component	Limitation	Severity	Impact
L1 8	4.7 Enrich	24h dedup window is in-memory, resets on restart	LOW	Potential re-enrichment after app restart

Table 9: Consolidated Limitations Tracker (18 items)

7.1 Severity Distribution

Severity	Count	Description
HIGH	2	Directly impacts production correctness; users will encounter errors
MEDIUM	8	Functionality gaps that degrade reliability or usability under real conditions
LOW	8	Code quality or edge-case issues; no immediate user impact but technical debt

Table 10: Limitation Severity Distribution

The 2 HIGH severity items (L1: CRM token refresh, L2: XLSX placeholder) represent the primary blockers to marking S7 as production-complete. The 8 MEDIUM items represent significant gaps that should be addressed before relying on these systems in a production environment. The 8 LOW items represent technical debt that can be addressed incrementally without blocking production deployment.

8. Cross-Cutting Architecture Patterns

8.1 Prisma Model Dependency Graph

The S7 components introduce or extend 6 Prisma models that form a coherent data graph. Understanding these dependencies is essential for database migration planning, backup strategies, and cascading delete behavior.

Model	Introduced By	Relations	Cascade Behavior
MergeRecord	4.4	Company (survivor + duplicate)	No cascade; orphaned on company delete
CRMConnection	4.5	CRMSyncLog[] (one-to-many)	Cascade delete removes all sync logs
CRMSyncLog	4.5	CRMConnection (many-to-one)	Cascade deleted with parent connection
DataExport	4.6	No child relations	Standalone; file cleanup required on delete
ImportTemplate	4.6	No child relations	Standalone; built-in templates are protected from deletion
EnrichmentJob	4.7	No child relations (references Company/Contact by ID string)	No FK constraint; orphan-safe

Table 11: S7 Prisma Model Dependencies

8.2 Error Handling Patterns

All four S7 components follow a consistent error handling philosophy: per-entity error isolation with best-effort progress. Rather than failing an entire batch when one entity encounters an error, the systems log the failure, continue processing remaining entities, and report partial results. This pattern is implemented in bulkMerge (continues after individual merge failures), syncFromCRM (try/catch per entity with CRMSyncLog failure entries), the import pipeline (per-row validation with status tracking), and the enrichment queue (per-item error capture with retry).

The API layer consistently returns HTTP 207 Multi-Status for operations that partially succeed, allowing clients to distinguish between complete success (200), partial success (207), and complete failure (500). This pattern is implemented in the CRM sync endpoint and should be extended to other bulk operations.

8.3 Audit Trail Coverage

All S7 components produce audit records through different mechanisms: MergeRecord (4.4) tracks every merge/skip action with actor, reason, and field-level detail. CRMSyncLog (4.5) tracks every synced entity with direction, action, duration, and errors. DataExport (4.6) tracks export job lifecycle with progress metrics. EnrichmentJob (4.7) tracks enrichment attempts with provider, retry count, confidence, and result data. Together, these four models provide comprehensive audit coverage for all data movement operations.

9. Assessment and Recommendations

9.1 Readiness Assessment

S7 delivers solid, well-tested foundational capabilities for the Revenue Intelligence data pipeline. The 124 tests provide good coverage of core algorithms and sync logic. The code is type-safe (zero TS errors), lint-clean (zero ESLint errors), and properly committed (de4acb15). The architectural patterns (provider abstraction, streaming, cursor pagination, idempotent operations) demonstrate production-grade engineering.

However, based on the 18 identified limitations (2 HIGH, 8 MEDIUM, 8 LOW), S7 should not yet be marked as "100% complete." The two HIGH severity items represent direct user-facing bugs: CRM token expiry will cause silent sync failures, and the XLSX placeholder will produce invalid files. Until these are resolved, the CRM integration and import/export features cannot be considered production-reliable.

9.2 Recommended Priority Order

Priority	Item	Effort	Impact
P0	L1: CRM automatic token refresh	2-3 hours	Prevents silent sync failures after token expiry
P0	L2: Real XLSX formatter (exceljs)	3-4 hours	Valid Excel output for import/export
P1	L3: Persist Clearbit usage counter to DB	1 hour	Prevents API quota overruns after restart
P1	L7: Fix getMergeHistory null dereference	30 min	Complete merge history data
P1	L9: Add multipart file upload for imports	4-5 hours	Enable large file imports in production
P2	L4: Consolidate dual enrichment systems	1-2 days	Remove confusion, prevent double-enrichment
P2	L5+L6: Implement webhook receivers + scheduled sync	2-3 days	Enable realtime and automated CRM sync
P2	L8: Integration tests for dedup merge functions	1-2 days	Regression safety for complex transactions
P3	L10-L18: All LOW items	Variable	Technical debt reduction

Table 12: Recommended Remediation Order

The estimated total effort to resolve all HIGH and MEDIUM items is approximately 3-5 developer-days. After P0 and P1 items are addressed, S7 can be confidently marked as production-complete for the core use cases. P2 items represent important hardening work that should follow in the next sprint cycle. P3 items can be addressed incrementally as technical debt.